

Relaciones entre clases

Asociación, herencia, polimorfismo, abstracción e interfaces

2.1 Asociación

La asociación es una relación entre clases cuyos objetos interactúan entre sí: tiene navegabilidad (uni/bidireccional) y multiplicidad (1-1, 1-N, N-N).

```
public class Estudiante {  
    private Curso[] cursos;  
    public void addCurso(Curso c) { /* ... */ }  
}  
public class Curso {  
    private Estudiante[] estudiantes;  
    public void addEstudiante(Estudiante e) { /* ... */ }  
}
```

Es bidireccional N-N: si solo Estudiante conociera sus Curso, sería unidireccional.

2.1 Agregación vs composición

	Agregación	Composición
¿Obligatoria?	No	Sí
¿Existe sin el contenedor?	Sí	No
Ejemplo	Curso y su Docente	Docente y sus datos de contacto

Curso ◊— Docente
(agregación)

Docente ◆— DatosContacto
(composición)

Si el objeto contenido no puede existir sin su contenedor, es composición; si puede existir de forma independiente, es agregación.

2.2 Herencia

La herencia crea una subclase a partir de una superclase, conservando sus miembros no privados e incorporando otros propios.

```
public class Persona {  
    protected String nombre;  
    public String mostrar() { return nombre; }  
}  
public class Estudiante extends Persona {  
    @Override  
    public String mostrar() { return "Estudiante " + super.mostrar(); }  
    public String saludar() { return "Hola, " + nombre; }  
}
```

2.2 Herencia (cont.)

Persona (superclase)

Estudiante (subclase, extends)

Estudiante hereda el atributo nombre y sobrescribe mostrar() con @Override, además de añadir su propio método saludar().

No confundas sobrescribir (override: misma firma heredada) con sobrecargar (overload: mismo nombre, distintos parámetros, sin relación con la herencia).

2.3 Polimorfismo

Una variable declarada del tipo de una superclase puede referenciar un objeto de cualquiera de sus subclases.

```
Persona estudiante = new Estudiante();      // asignación polimorfa
System.out.println(estudiante.mostrar());  // ejecución polimorfa
System.out.println(((Estudiante) estudiante).saludar()); // casting
```

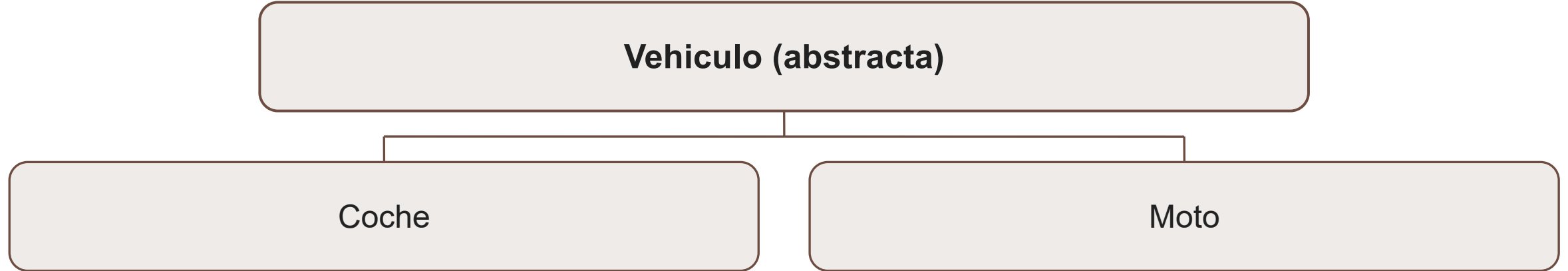
`estudiante.saludar()` no compila sin casting: el compilador solo ve el tipo declarado (`Persona`), no el objeto real.

2.4 Clases abstractas

Una clase abstracta no se puede instanciar. Puede declarar métodos abstractos, sin cuerpo, que las subclases están obligadas a implementar.

```
public abstract class Vehiculo {  
    protected String marca;  
    public abstract double calcularConsumo(); // cada subclase decide  
    public String arrancar() { return marca + " arrancando..."; }  
}  
public class Coche extends Vehiculo {  
    @Override public double calcularConsumo() { return 6.5; }  
}
```

2.4 Clases abstractas (cont.)



`new Vehiculo()` no compilaría: solo se pueden crear objetos de subclases concretas, como `Coche` o `Moto`.

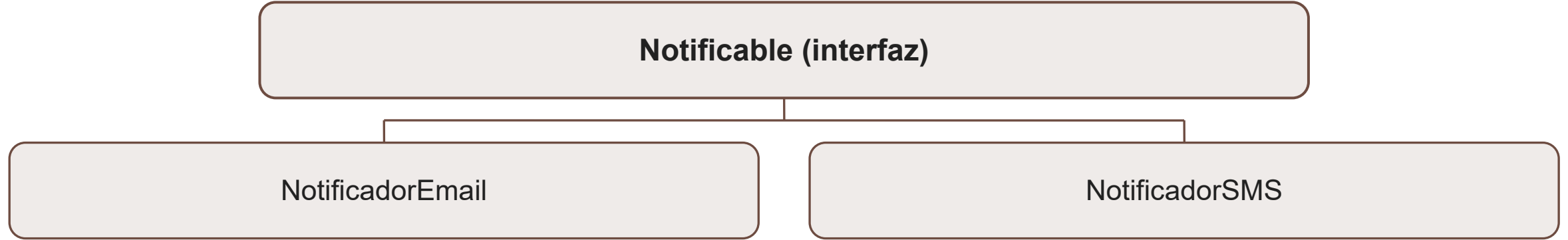
Vehiculo mezcla los dos tipos de métodos: `calcularConsumo()` es abstracto y `arrancar()` ya viene implementado. Una clase abstracta no está obligada a ser 100% abstracta.

2.5 Interfaces

Una interfaz define un contrato de comportamiento. A diferencia de la herencia, la pueden implementar clases sin ninguna relación entre sí.

```
public interface Notificable {  
    void enviar(String mensaje);  
}  
  
public class NotificadorEmail implements Notificable {  
    @Override public void enviar(String mensaje) { /* ... */ }  
}  
  
public class NotificadorSMS implements Notificable {  
    @Override public void enviar(String mensaje) { /* ... */ }  
}
```

2.5 Interfaces (cont.) — métodos default



```
public interface Notificable {
    void enviar(String mensaje);
    default void enviarConAviso(String mensaje) { // ya trae implementación
        enviar(mensaje);
    }
}
```

Java no permite herencia múltiple de clases, pero sí implementar varias interfaces a la vez.

2.6 Interfaces funcionales

Una interfaz es funcional si declara un único método abstracto. Comparable define compareTo, que compara this con otro objeto o.

```
public class Estudiante implements Comparable<Estudiante> {  
    @Override  
    public int compareTo(Estudiante o) {  
        return o.edad - this.edad; // ordenación descendente por edad  
    }  
}
```

Valor devuelto	Significado
Negativo	this es menor que o
Cero	this es igual a o
Positivo	this es mayor que o

Resumen del apartado

Relación	¿Obligatoria?	¿Cuántas puede tener una clase?	Palabra clave
Asociación / Agregación	No	Varias	atributo normal
Composición	Sí	Varias	atributo creado dentro de la clase
Herencia	—	Como mucho una superclase	extends
Interfaz	—	Varias a la vez	implements